Concurrencia en Android: corrutinas, dispatchers y ciclo de vida

Proyecto de partida

- Para esta presentación vamos a comenzar de un proyecto base.
- Para disponer del proyecto base tenemos que:
  - Crear un proyecto nuevo con la plantilla Empty Activity en Android Studio.
  - En el fichero MainActivity.kt del proyecto integrar el código del MainActivity.kt dispuesto en una diapositiva posterior.
  - Crear el fichero de kotlin PantallaPrincipal.kt y en él integrar el código del PantallaPrincipal.kt dispuesto en una diapositiva posterior.
  - Probar a ejecutar el proyecto y ver que funciona correctamente.
  - Modificar el código del fichero PantallaPrincipal.kt para que el código sea como el dispuesto en en una diapositiva posterior.

MainActivity.kt

```
package eus.fptxurdinaga.psp.p05_00

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            PantallaPrincipal()
        }
    }
}
```

PantallaPrincipal.kt

```
package eus.fptxurdinaga.psp.p05_00

import androidx.compose.foundation.layout.Column
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable

@Composable
fun PantallaPrincipal() {

    Column {
        Text("PSP - Concurrencia en Android")
    }
}
```

Ejecutar el proyecto

PantallaPrincipal.kt (1/2)

```
package eus.fptxurdinaga.psp.p05_00

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
```

PantallaPrincipal.kt (2/2)

```
@Composable
fun PantallaPrincipal() {
    var pulsaciones by remember {
        mutableStateOf(0)
    }
    Column(
        modifier = Modifier.padding(40.dp)
    ) {
        Text("Pulsaciones: $pulsaciones")
        Button(onClick = {
            println(Thread.currentThread().name)
            Thread.sleep(3000)
        }) {
            Text("Bloquear 3 s")
        }
        Button(onClick = {
            pulsaciones++
        }) {
            Text("Probar respuesta")
        }
    }
}
```

Concurrencia en Android

corrutinas, dispatchers y ciclo de vida

Ya sabemos

Presentación de "Concurrencia y procesos: del programa a ProcessBuilder"
¿merece la pena la concurrencia?

↓

Presentación de "Hilos JVM con Thread: creación, ejecución y ciclo de vida"
Thread · bloqueo · espera · cancelación

↓

Presentación de "Datos compartidos, condiciones de carrera y synchronized"
estado compartido · carreras · synchronized

Hoy

Android Main

↓

bloqueo ≠ suspensión

↓

corrutina ≠ hilo

↓

launch · estructura

↓

dispatcher · propietario

↓

Compose / LaunchedEffect

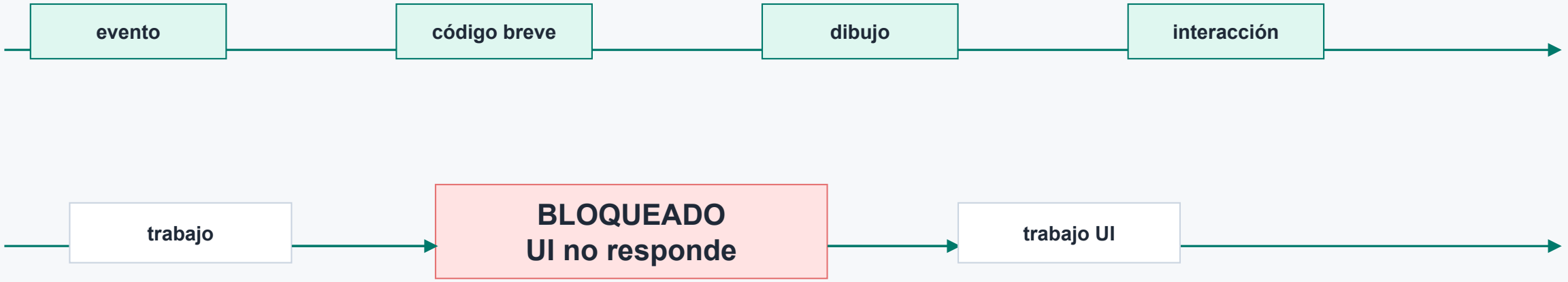
Resultado final

Elegir, implementar y justificar concurrencia en una app Compose sencilla sin perder la competencia explícita con Thread.

concurrencia y procesos → Thread → estado compartido y sincronización → Android y corrutinas

Main: el hilo que no queremos bloquear (1/2)

Main debe seguir suficientemente disponible



No bloquear innecesariamente Main.

Pregunta predictiva
Durante esos tres segundos, ¿qué hilo está esperando?

No necesitamos estudiar Looper ni MessageQueue: el objetivo visual es la responsividad observable.

Main: el hilo que no queremos bloquear (2/2)

Dentro del Column ya proporcionado:

```
var pulsaciones by remember {
    mutableStateOf(0)
}

Text("Pulsaciones: $pulsaciones")

Button(onClick = {
    println(
        Thread.currentThread().name
    )
    Thread.sleep(3000)
}) {
    Text("Bloquear 3 s")
}

Button(onClick = {
    pulsaciones++
}) {
    Text("Probar respuesta")
}
```

Demostración

1. Predecir.
2. Pulsar “Bloquear 3 s”.
3. Intentar “Probar respuesta”.
4. Observar la respuesta diferida.
5. Mirar el nombre del hilo.

Resultado esperado

La interfaz queda temporalmente sin capacidad normal de respuesta.

Interpretación

El problema no es “Android es lento”; Main está ocupado esperando.

Compose mínimo: estado observable

Solo el Compose que PSP necesita

```
@Composable
fun Contador() {
    var contador by remember {
        mutableStateOf(0)
    }

    Text("Contador: $contador")

    Button(onClick = {
        contador++
    }) {
        Text("Sumar")
    }
}
```

remember

Conserva un objeto en esa posición de la composición mientras permanezca en ella.

mutableStateOf(...)

Crea estado observable por Compose.

cambio de estado → **Compose puede programar recomposición** → **UI actualizada**

NO significa

remember ≠ synchronized

MutableState ≠ lock

remember ≠ persistencia general

Pregunta: ¿remember resuelve por sí mismo una carrera entre dos hilos? → No.

Examen histórico: tres capas diferentes

Compose + Thread + sincronización

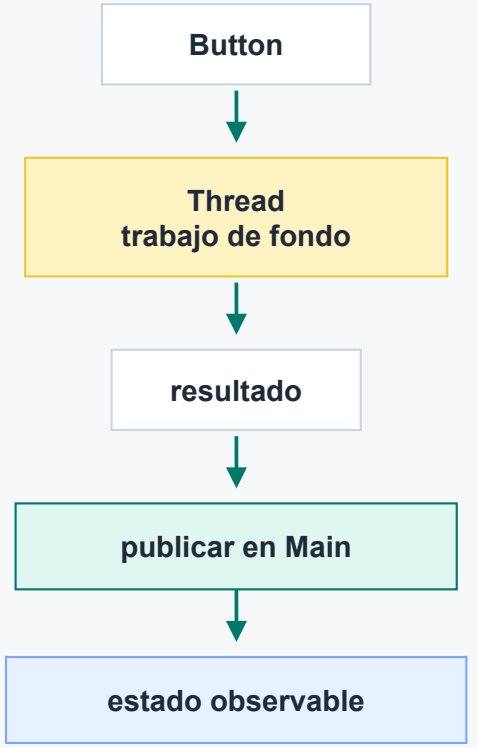
Pregunta	Herramienta
¿Qué valor observa la UI?	remember / MutableState
¿Qué ejecuta trabajo explícitamente en otro hilo?	Thread
¿Cómo protegemos una sección crítica si hace falta?	synchronized(lock)

En la demostración el proyecto ya proporciona:

```
postToMain {
    texto = resultado
}
```

postToMain { ... }
es infraestructura auxiliar (disponible en la siguiente diapositiva),
no contenido de PSP.

Puente corregido



No presentamos un Thread modificando arbitrariamente estado UI desde cualquier contexto como patrón general.

AndroidUtils.kt

```
package eus.fptxurdinaga.psp.p05_00


import android.os.Handler
import android.os.Looper


fun postToMain(block: () -> Unit) {
    Handler(Looper.getMainLooper()).post(block)
}
```

Una corrutina no es un hilo

Thread sigue existiendo y añadimos otra abstracción

No decimos

Thread está obsoleto

corrutina = hilo ligero

launch = crear Thread

Modelo de esta presentación

- Una corrutina es una computación suspendible.
- Puede progresar concurrentemente con otras.
- En JVM/Android su código se ejecuta utilizando hilos.

NO existe la regla

1 corrutina = 1 hilo

Modelo visual



Durante una suspensión de una corrutina, un hilo puede atender otro trabajo.

No se garantiza mismo hilo ni hilo distinto al reanudar: depende del contexto.

suspend, sleep y delay

Suspender no es bloquear

```
suspend fun avanzar() {
    delay(1000)
}
```

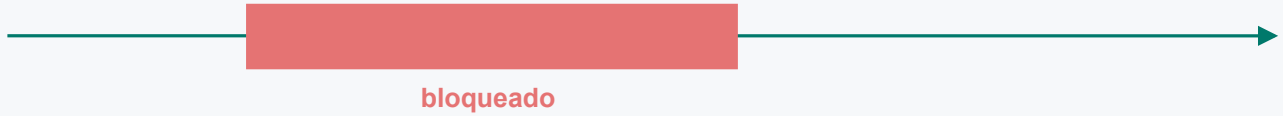
suspend permite que la función tenga puntos de suspensión.

No significa

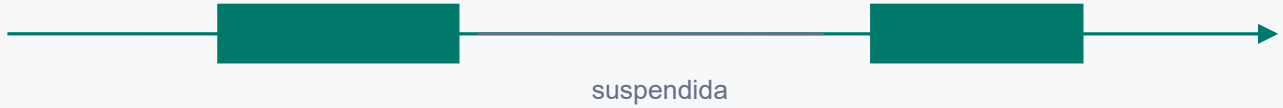
- crear corrutina
- crear Thread
- segundo plano
- Dispatchers.IO
- concurrencia automática

Adición de dependencias en la próxima diapositva

Thread.sleep(1000)



delay(1000)



Bloquear un hilo ≠ suspender una corrutina

delay() tampoco cambia automáticamente el dispatcher.

suspend, sleep y delay

- En libs.versions.toml, añadir en [versions]:
  - coroutines = “1.11.0”
- En libs.versions.toml, añadir en [libraries]:
  - kotlinx-coroutines-android = { group = "org.jetbrains.kotlinx", name = "kotlinx-coroutines-android", version.ref = "coroutines" }
- En el build.gradle.kts del módulo app, dentro de dependencies {…}, añadir:
  - implementation(libs.kotlinx.coroutines.android)
- Pulsar Sync Project with Gradle files

CoroutineScope, launch y Job

¿Desde dónde lanzamos el trabajo?

```
suspend fun tareaSuspendible() {
    delay(1000)
    println("fin")
}

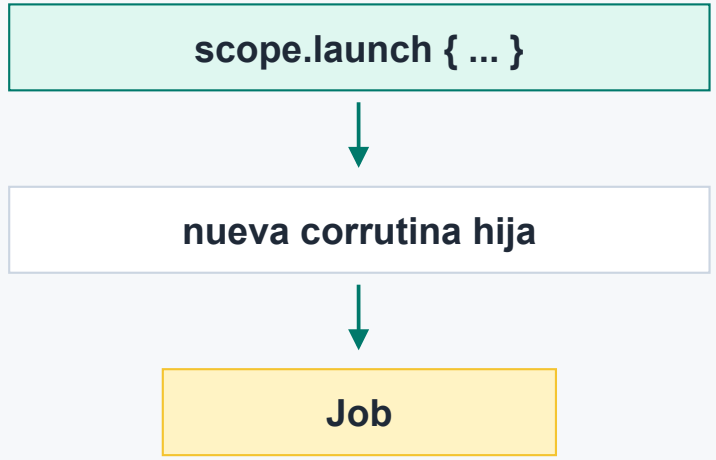
fun iniciar(
    scope: CoroutineScope
): Job =
    scope.launch {
        tareaSuspendible()
    }
```

Ejemplo en la próxima diapositiva

launch ≠ Thread()

CoroutineScope

Proporciona contexto y estructura para corrutinas relacionadas.



- permite que el código llamante continúe
- no significa crear un nuevo Thread

Job representa/controla la vida de ese trabajo al nivel que necesitamos.

EjemplosCorrutinas.kt

```
package eus.fptxurdinaga.psp.p05_00


import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Job
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch


suspend fun tareaSuspendible() {
    delay(1000)
    println("fin")
}


fun iniciar(scope: CoroutineScope): Job =
    scope.launch {
        tareaSuspendible()
    }
```

suspend no implica concurrencia

Secuencial frente a concurrente

```
suspend fun tareaA() {
    delay(1000)
}

suspend fun tareaB() {
    delay(1000)
}
```

Ejemplo en la próxima diapositiva

No garantiza

paralelismo físico

orden exacto

hilos concretos

Secuencial

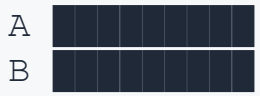
```
tareaA()
tareaB()
```



B se llama cuando A ha retornado.

Concurrente

```
launch { tareaA() }
launch { tareaB() }
```



A y B pueden progresar durante el mismo intervalo.

suspend permite suspensión. launch crea una corrutina concurrente.

EjemplosCorrutinas.kt

```
package eus.fptxurdinaga.psp.p05_00


import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Job
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch


suspend fun tareaSuspendible() {
    delay(1000)
    println("fin")
}


fun iniciar(scope: CoroutineScope): Job =
    scope.launch {
        tareaSuspendible()
    }


suspend fun tareaA() {
    delay(1000)
}


suspend fun tareaB() {
    delay(1000)
}
```

Concurrencia estructurada: nadie debería perder a sus tareas

El trabajo concurrente necesita propietario y estructura

Problema

lanzo tareas

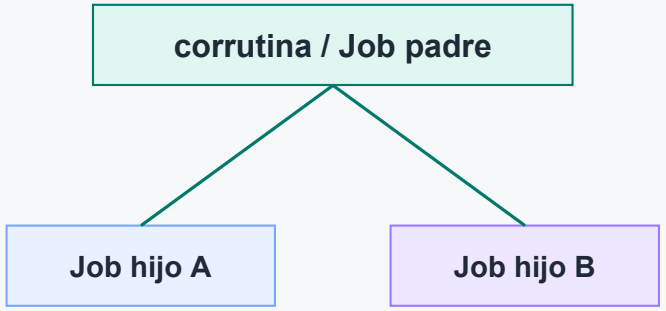
↓

pierdo quién las posee

↓

¿quién espera?
¿quién cancela?
¿cuándo terminan?

Estructura




```
suspend fun ejecutarDos() =
    coroutineScope {
        launch { tareaA() }
        launch { tareaB() }
    }
```

- crea un scope hijo basado en el contexto actual
- no retorna hasta que terminan el bloque y sus hijos
- espera mediante suspensión, no bloqueando necesariamente un hilo

Ejemplo en la próxima diapositiva

Trabajo concurrente con propietario estructurado.

EjemplosCorrutinas.kt (1/2)

```
package eus.fptxurdinaga.psp.p05_00

import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Job
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
import kotlinx.coroutines.coroutineScope

suspend fun tareaSuspendible() {
    delay(1000)
    println("fin")
}

fun iniciar(scope: CoroutineScope): Job =
    scope.launch {
        tareaSuspendible()
    }
```

EjemplosCorrutinas.kt (2/2)

```
suspend fun tareaA() {
    delay(1000)
}

suspend fun tareaB() {
    delay(1000)
}

suspend fun ejecutarDos() =
    coroutineScope {
        launch { tareaA() }
        launch { tareaB() }
    }
```

De secuencial a concurrente estructurado (1/2)

El proyecto base proporciona participar(...):

```
suspend fun participar(
    nombre: String,
    pausaMs: Long
) {
    repeat(5) { vuelta ->
        delay(pausaMs)
        println("$nombre: ${vuelta + 1}")
    }
}
```

Versión secuencial

```
suspend fun carreraSecuencial() {
    participar("A", 500)
    participar("B", 300)
}
```



Primero termina A.
Después empieza B.

Ejemplo en la próxima diapositiva

suspendible ≠ concurrente por sí mismo

EjemplosCorrutinas.kt (1/3)

```
package eus.fptxurdinaga.psp.p05_00

import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Job
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
import kotlinx.coroutines.coroutineScope

suspend fun tareaSuspendible() {
    delay(1000)
    println("fin")
}

fun iniciar(scope: CoroutineScope): Job =
    scope.launch {
        tareaSuspendible()
    }
```

EjemplosCorrutinas.kt (2/3)

```
suspend fun tareaA() {
    delay(1000)
}

suspend fun tareaB() {
    delay(1000)
}

suspend fun ejecutarDos() =
    coroutineScope {
        launch { tareaA() }
        launch { tareaB() }
    }
```

EjemplosCorrutinas.kt (3/3)

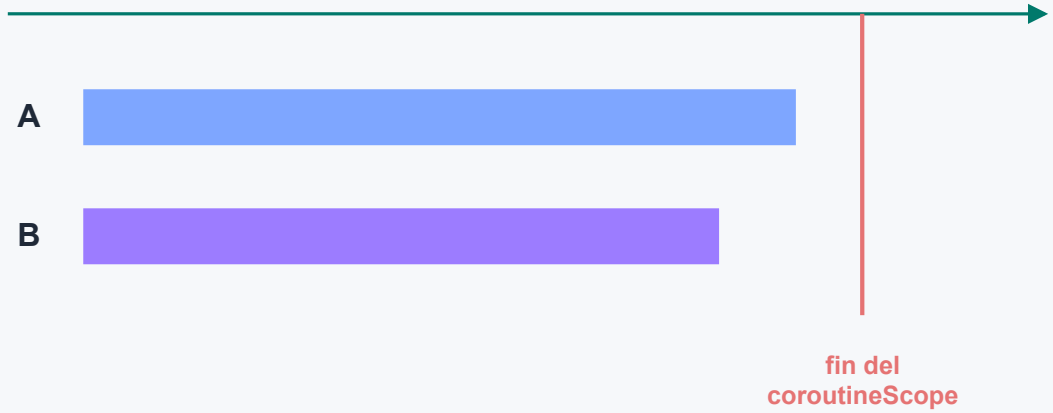
```
suspend fun participar(
    nombre: String,
    pausaMs: Long
) {
    repeat(5) { vuelta ->
        delay(pausaMs)
        println("$nombre: ${vuelta + 1}")
    }
}

suspend fun carreraSecuencial() {
    participar("A", 500)
    participar("B", 300)
}
```

De secuencial a concurrente estructurado (2/2)

Versión concurrente estructurada

```
suspend fun carreraConcurrente() =
    coroutineScope {
        launch {
            participar("A", 500)
        }
        launch {
            participar("B", 300)
        }
    }
```




```
carreraConcurrente()
println("Carrera terminada")
```

Garantía relevante

“Carrera terminada” se ejecuta después de completar A y B.

Ejemplo en la próxima diapositiva

Transformación guiada
secuencial → coroutineScope + 2 launch

EjemplosCorrutinas.kt (1/3)

```
package eus.fptxurdinaga.psp.p05_00

import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Job
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
import kotlinx.coroutines.coroutineScope

suspend fun tareaSuspendible() {
    delay(1000)
    println("fin")
}

fun iniciar(scope: CoroutineScope): Job =
    scope.launch {
        tareaSuspendible()
    }
```

EjemplosCorrutinas.kt (2/3)

```
suspend fun tareaA() {
    delay(1000)
}

suspend fun tareaB() {
    delay(1000)
}

suspend fun ejecutarDos() =
    coroutineScope {
        launch { tareaA() }
        launch { tareaB() }
    }
```

EjemplosCorrutinas.kt (3/3)

```
suspend fun participar(
    nombre: String,
    pausaMs: Long
) {
    repeat(5) { vuelta ->
        delay(pausaMs)
        println("$nombre: ${vuelta + 1}")
    }
}


suspend fun carreraSecuencial() {
    participar("A", 500)
    participar("B", 300)
}


suspend fun carreraConcurrente() =
    coroutineScope {
        launch {
            participar("A", 500)
        }
        launch {
            participar("B", 300)
        }
    }
```

Prueba de secuencial y de concurrente

- Vamos a añadir código expresamente para probar la secuencial y el concurrente.
- En EjemplosCorrutinas.kt añadimos lo que viene en la próxima diapositiva.
- En PantallaPrincipal.kt añadimos lo que viene dentro de dos diapositivas.
- Ejecutar, abrir LogCat y probar los dos botones añadidos.

Añadir a EjemplosCorrutinas.kt

```
import kotlinx.coroutines.runBlocking
```

```
fun ejecutarDemoSecuencial() {
    Thread {
        runBlocking {
            println("=== SECUENCIAL ===")
            carreraSecuencial()
            println("Carrera terminada")
        }
    }.start()
}


fun ejecutarDemoConcurrente() {
    Thread {
        runBlocking {
            println("=== CONCURRENTE ===")
            carreraConcurrente()
            println("Carrera terminada")
        }
    }.start()
}
```

Añadir a PantallaPrincipal.kt

```
Button(
    onClick = {
        ejecutarDemoSecuencial()
    }
) {
    Text("Carrera secuencial")
}
Button(
    onClick = {
        ejecutarDemoConcurrente()
    }
) {
    Text("Carrera concurrente")
}
```

Cancelación: el mismo principio cooperativo

Cancelar trabajo no significa “matar un hilo”

La presentación de "Hilos JVM con Thread: creación, ejecución y ciclo de vida" · Thread

interrupt()

↓

operación / código coopera

↓

fin

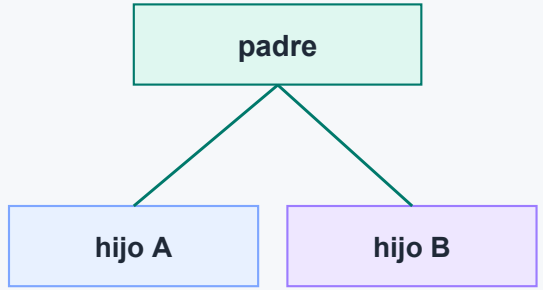
Esta presentación · Corrutina

```
val job = scope.launch {
    while (true) {
        delay(1000)
        println("tick")
    }
}

job.cancel()
```

cancelación
↓
delay detecta la cancelación
↓
la corrutina termina

Estructura



cancelar padre
↓
cancela sus hijos

cancelar hijo A
≠
cancelar automáticamente todos sus hermanos

Ejemplo en la próxima diapositiva

EjemplosCorrutinas.kt (1/3)

```
package eus.fptxurdinaga.psp.p05_00

import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Job
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
import kotlinx.coroutines.coroutineScope

suspend fun tareaSuspendible() {
    delay(1000)
    println("fin")
}

fun iniciar(scope: CoroutineScope): Job =
    scope.launch {
        tareaSuspendible()
    }
```

EjemplosCorrutinas.kt (2/3)

```
suspend fun tareaA() {
    delay(1000)
}


suspend fun tareaB() {
    delay(1000)
}


suspend fun ejecutarDos() =
    coroutineScope {
        launch { tareaA() }
        launch { tareaB() }
    }


suspend fun participar(
    nombre: String,
    pausaMs: Long
) {
    repeat(5) { vuelta ->
        delay(pausaMs)
        println("$nombre: ${vuelta + 1}")
    }
}
```

EjemplosCorrutinas.kt (3/3)

```
suspend fun carreraSecuencial() {
    participar("A", 500)
    participar("B", 300)
}

suspend fun carreraConcurrente() =
    coroutineScope {
        launch {
            participar("A", 500)
        }
        launch {
            participar("B", 300)
        }
    }

fun iniciarTicks(scope: CoroutineScope): Job =
    scope.launch {
        while (true) {
            delay(1000)
            println("tick")
        }
    }
```

Scope y dispatcher no responden a la misma pregunta

Dos preguntas distintas

1. ¿Quién posee el trabajo?
¿Cuánto debe vivir?

2. ¿Dónde debe ejecutarse?

CoroutineScope
Job
ciclo de vida

CoroutineDispatcher

scope
→ propiedad · vida · estructura

dispatcher
→ contexto de ejecución

Matiz técnico

```
CoroutineScope
└── CoroutineContext
    └── puede incluir un dispatcher
```

responsabilidades distintas ≠ universos desconectados

Main, IO, Default y withContext

Elegir contexto según la naturaleza del trabajo

Trabajo	Dispatcher orientativo
interacción/actualización UI y trabajo breve	Dispatchers.Main
E/S bloqueante	Dispatchers.IO
cálculo intensivo de CPU	Dispatchers.Default

withContext

```
suspend fun cargar(): String =
    withContext(Dispatchers.IO) {
        leerBloqueante()
    }
```

```
El proyecto proporciona:
fun leerBloqueante(): String
```

cambia el contexto del bloque

↓

suspende hasta que termina

↓

devuelve resultado

↓

continúa en el contexto anterior

No es otro launch

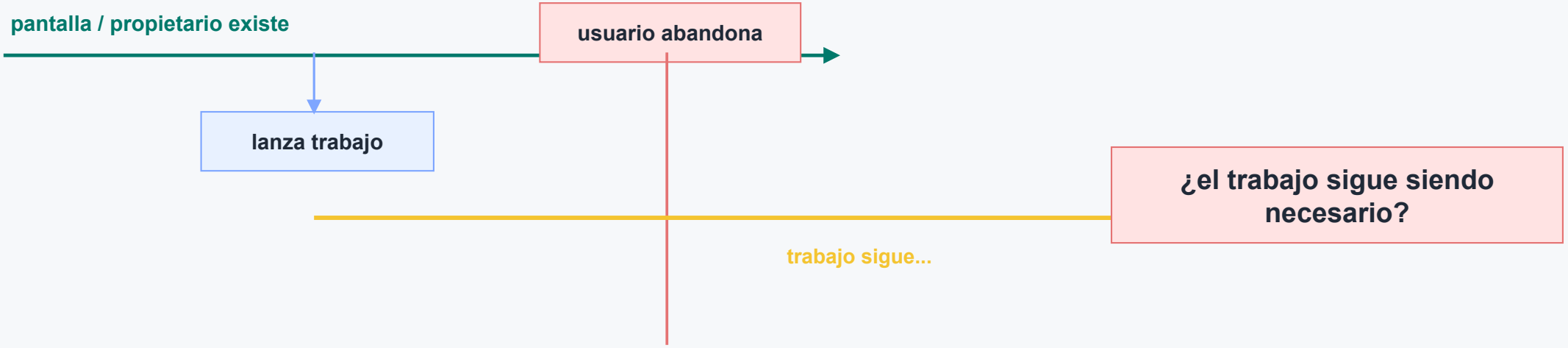
launch
→ nueva corrutina hija
→ Job

withContext
→ bloque + resultado
→ espera suspendible

La tabla es guía de razonamiento, no clasificación infalible de cualquier API.

El problema del ciclo de vida: ¿quién sigue necesitando la tarea?

Una tarea puede durar más que quien la pidió



- podemos gastar recursos inútilmente
- conservar trabajo que ya nadie necesita
- intentar publicar un resultado donde ya no corresponde

Todo trabajo concurrente debe tener un propietario y una vida útil coherentes con la necesidad que lo originó.

Propietarios que reconoceremos

ViewModel | **LifecycleOwner** | **composición**

viewModelScope y lifecycleScope (1/2)

Trabajo ligado al ViewModel

```
class EmpresaViewModel : ViewModel() {

    var texto by mutableStateOf("Sin datos")
        private set

    fun cargarDatos() {
        viewModelScope.launch {
            texto =
                withContext(Dispatchers.IO) {
                    leerBloqueante()
                }
        }
    }
}
```

El proyecto proporciona:
`fun leerBloqueante(): String`

viewModelScope

→ **pertenece al ViewModel**
→ **se cancela al limpiarse**
→ **parte de Main**

Error a evitar

viewModelScope ≠ hilo de fondo

La operación bloqueante se mueve con:
`withContext(Dispatchers.IO)`

viewModelScope y lifecycleScope (2/2)

Trabajo ligado a un LifecycleOwner

```
// Dentro de una Activity/Fragment
lifecycleScope.launch {
    tareaSuspendible()
}
```

lifecycleScope

→ pertenece al LifecycleOwner
→ se cancela cuando se destruye

La pregunta es quién posee el trabajo y cuánto debe vivir.

No son sinónimos

¿pertenece al ViewModel?

→ viewModelScope

¿pertenece a este LifecycleOwner?

→ lifecycleScope

Propiedad y vida

```
ViewModel
└── viewModelScope
```

```
LifecycleOwner
└── lifecycleScope
```

LaunchedEffect: trabajo ligado a composición

Compose también puede ser propietario de un efecto

```
LaunchedEffect(clave) {
    // código suspendible
}
```

entra en composición → **inicia una corrutina**

sale de composición → **cancela esa corrutina**

Cambio de clave

clave A → clave B

↓

cancelar ejecución A

↓

iniciar ejecución B

Propietario	Mecanismo
ViewModel	viewModelScope
LifecycleOwner	lifecycleScope
composición	LaunchedEffect

Recomposición por sí sola
≠
reinicio automático

Las claves determinan cuándo debe reiniciarse el efecto mientras permanece en composición.

```
LaunchedEffect(idCliente) {
    cargarCliente(idCliente)
}
```

Si idCliente cambia: ejecución anterior cancelada → nueva ejecución.

Segundero: suspensión y composición (1/2)

Mismo problema, otra gestión del trabajo

Dentro del Column ya proporcionado:

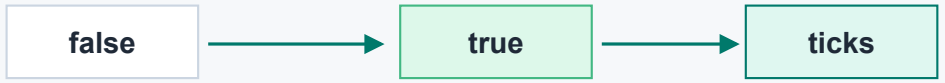
```
var segundos by remember {
    mutableStateOf(0)
}

var activo by remember {
    mutableStateOf(false)
}

LaunchedEffect(activo) {
    if (!activo) {
        return@LaunchedEffect
    }

    while (true) {
        delay(1000)
        segundos++
    }
}
```

Cuando cambia activo



ejecución anterior → cancelada
nueva clave → nueva ejecución

Comparación

La presentación de "Hilos JVM con Thread: creación, ejecución y ciclo de vida"	Esta presentación
Thread + sleep + interrupt	corrutina + delay + clave + cancelación

El contenido nuevo es la gestión del trabajo, no contar segundos.

Segundero: suspensión y composición (2/2)

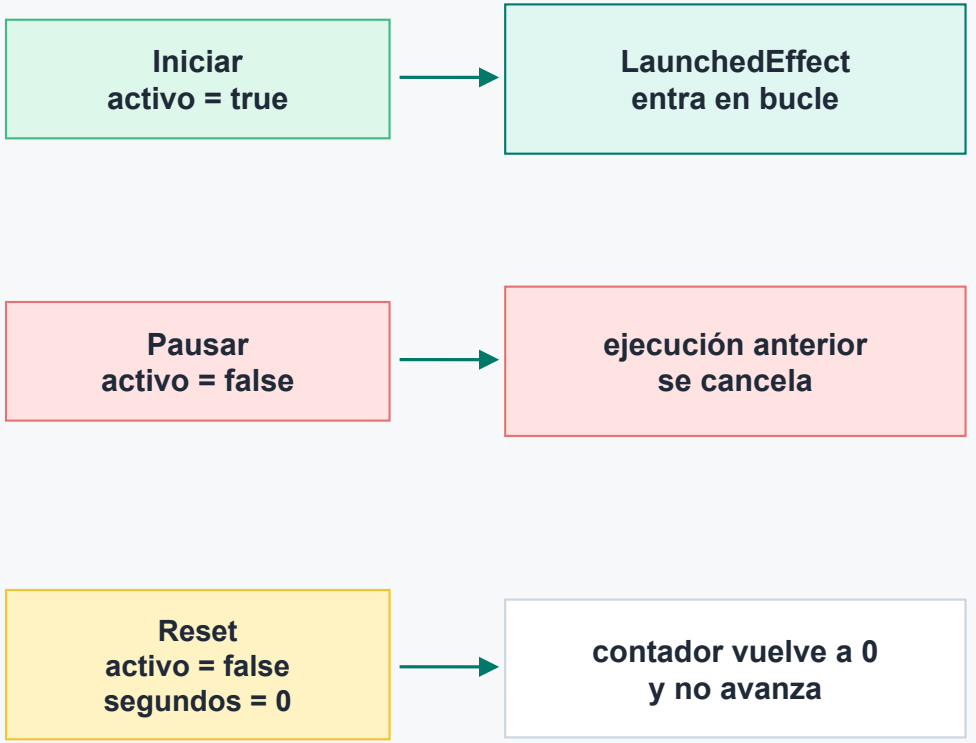
Botones y resultado observable

```
Button(
    onClick = { activo = true }
) {
    Text("Iniciar")
}

Button(
    onClick = { activo = false }
) {
    Text("Pausar")
}

Button(
    onClick = {
        activo = false
        segundos = 0
    }
) {
    Text("Reset")
}

Text("Segundos: $segundos")
```



¿Qué Thread.sleep necesitamos aquí? → Ninguno.

Micropráctica
completar delay(1000) y segundos++ · ejecutar · pausar · resetear · comparar con bloqueo

Elegir mecanismo antes de programar

Secuencial, proceso, Thread o corrutina

Situación	Opción razonable
trabajo pequeño y dependiente	secuencial
ejecutar herramienta/programa externo	proceso
hilo JVM explícito o lo exige la evaluación	Thread
concurrencia/asíncronía estructurada en Kotlin/Android	corrutina

No es un ranking

```
corrutina
    >
Thread
    >
proceso
    >
secuencial
```

Eso sería incorrecto.

Antes de decidir

- ¿Qué trabajo existe?
- ¿Necesita realmente concurrencia?
- ¿Qué mecanismo?
- ¿Existe estado compartido?
- ¿Quién posee el trabajo?
- ¿Cuánto debe vivir?
- ¿Dónde debe ejecutarse?
- ¿Cómo comprobaré que funciona?

La respuesta buena incluye una justificación, no solo el nombre de una API.

Decidir, justificar y comprobar

Transferencia al reto

1. ¿Hace falta concurrencia?
2. ¿Qué tipo de trabajo es?
3. ¿Qué mecanismo?
4. ¿Qué dispatcher, si procede?
5. ¿Quién lo posee?
6. ¿Cuánto vive?
7. ¿Hay estado compartido?
8. ¿Cómo lo comprobarías?

Formato de respuesta

"Usaría _____ porque _____.
Lo ligaría a _____.
Lo comprobaría mediante _____."

Funcionalidad plausible	Decisión
Mostrar inmediatamente el nombre elegido	?
Cargar información mediante E/S bloqueante	?
Realizar un cálculo local intensivo	?
Actualizar un progreso visible	?
Trabajo que deja de importar al desaparecer la pantalla	?
Ejercicio individual que exige dos Thread	?

Objetivo del reto

correcto funcionamiento
+
concurrencia justificada
+
sin bloqueos perjudiciales

Cierre de las presentaciones de la evaluación 1

Lo que debe quedar operativo

Procesos JVM y concurrencia

- programa – proceso - hilo
- concurrencia ≠ paralelismo
- ProcessBuilder
- start() / isAlive
- waitFor()
- stdin / stdout / stderr
- proceso padre e hijos
- lanzar antes de esperar

Hilos JVM y sincronización

- Thread
- run() / start()
- sleep() / join()
- interrupt()
- nuevo Thread para reiniciar
- estado compartido
- synchronized
- Compose mínimo
- remember ≠ sincronización

Android moderno

- Main disponible
- corrutina ≠ hilo
- suspend
- sleep ≠ delay
- launch
- Job
- coroutineScope
- cancelación
- scope ≠ dispatcher
- Main / IO / Default
- withContext
- viewModelScope
- lifecycleScope
- LaunchedEffect

Recorrido completo

Presentación de "Concurrencia y procesos: del programa a ProcessBuilder"
DECIDIR

↓

Presentación de "Hilos JVM con Thread: creación, ejecución y ciclo de vida"
EJECUTAR

↓

Presentación de "Datos compartidos, condiciones de carrera y synchronized"
PROTEGER

↓

Esta presentación
ESTRUCTURAR
EN ANDROID

Concurrencia correcta = decisión + mecanismo + vida + contexto + comprobación

No se trata de usar más concurrencia, sino de usar exactamente la concurrencia necesaria y demostrar que el programa completo funciona correctamente.